

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Schumacher, J. M. *Introduction to Financial Derivatives*. Open Press TiU.

<https://research.tilburguniversity.edu/en/publications/introduction-to-financial-derivatives>

- **Required pages:** Chapter 2, Section 2.1
- **Estimated reading time:** 30 minutes

2. finCampus Lecture Hall. "Pricing an American Option: 3 Period Binomial Tree Model." *YouTube*, 26 May 2013,

<https://www.youtube.com/watch?v=35n7TICJbLc>.

- **Required pages:** Entire video
- **Estimated reading time:** 15 minutes

NOTE: Content presented in the Required Readings and Lesson Notes will be tested in all assignments, including Quizzes.

LESSON 1

INTRO TO AMERICAN OPTIONS

Derivative Pricing - Module 2



Outline

- ▶ American Derivatives: Call and Put Options
- ▶ Binomial Model for American Options
- ▶ Optimality of Early-Exercise

American Derivatives: Calls & Puts

We already know the basics of pricing **European options** using the binomial model. Now, we are going to use this model to price **American options**.

⇒ How are American options different from European ones?

- ▶ **American options** allow the buyer to exercise the option at any point in time.
- ▶ This important characteristic has further derived implications in real markets:
 - Flexibility to the buyer?
 - Risk to the seller?
 - Hedging (delta) exposure?
 - Over-The-Counter (OTC) vs. Exchange-traded?
 - Popularity?
 - Higher premiums (option prices)?

In this module, we will discuss all these implications further expanding on the Binomial tree pricing methodology we are already familiar with!

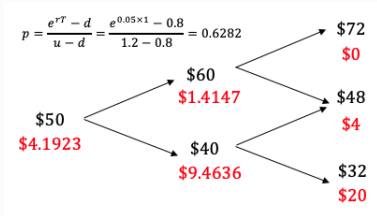
Binomial Model for American Options

Regardless of early exercise, **the value at maturity of an American option is equal to its European counterpart** → For example, for a Call option = $(S_T - K)^+$

However, to establish the **value (and price) of an American option** we will have to take into account:

1. The value of European counterpart (no early-exercise)
2. The payoff from early exercise

Let's compare the two. Picture a European Put (values in red) with:
 $u = 1.2$, $d = 0.8$, $T = 2 = N$, $dt = 1$, $r = 5\%$, $K = \$52$ and $S_0 = \$50$.

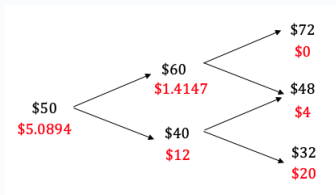


Binomial Model for American Options

The latter will give us the value of (1) European option (no early exercise).

Value of the American option will be the maximum between (1) European counterpart and (2) payoff from early-exercise.

To see this, let's analyze the case for an American Put with same characteristics:



- Now, in the upward path at $t = 1, \{u\}$, there is no benefit from early exercise, so the payoff remains the same as the European option.
- But in the downward path at $t = 1, \{d\}$, it is optimal to exercise early since the payoff (\$12) is higher than European value (\$9.46).
- In case of early exercise, the premium (price) for the American option is greater than European one.

Is Early Exercise Optimal?

In the latter example, we found it optimal (at node $\{d\}$) to exercise the Put before maturity $\Rightarrow$ When is this the case?

- For the American Call, early exercise is never optimal:

Consider two portfolios:

1. one American Call (c), and $Ke^{-r(T-t)}$ in cash (to pay for the strike).
2. one share of stock at price S_t .

If we exercise early at time $\tau < T$, the payoff will be:

$$\text{Portfolio 1} = (S_\tau - K) + Ke^{-r(T-\tau)} < S_\tau$$

If we exercise at maturity, $\tau = T$, the payoff will be:

$$\text{Portfolio 1} = \max(S_T - K, 0) + K \geq S_T$$

Value of the option is maximized holding until maturity $\rightarrow$ early exercise not optimal.

$\rightarrow$ Note that this is the case for American Calls on no-dividend-paying stocks

Is Early Exercise Optimal?

Early exercise is never optimal for call options on non-dividend-paying stocks... what about Put options?

- ▶ For the American put, early exercise can sometimes be optimal:

This is because, for deep in-the-money options with early exercise, you may earn more from the interest on the money than you stand to gain when price, S_t , is close to 0:

Suppose you hold a put option with $K = \$5$ and stock price, $S_t = 0$, at *time* $t < T$

- ▶ If you exercise now, you obtain an immediate gain of \$5 that can earn r for a period $T - t$.
- ▶ If you wait, chances are the gain from exercise is lower than \$5, but it will never be more than \$5 because the stock price cannot decrease below 0.

Hence, **early exercise can sometimes be optimal** for put options.

⇒ For the case of dividend-paying stocks, there may be cases where early exercise is optimal, for both call and put options.

Summary of Lesson 1

In lesson 1 we have covered a bunch of important concepts:

- ▶ American options payoff and pricing under binomial framework
- ▶ The trade-off between early exercise and holding until maturity
- ▶ When is early-exercise optimal?

⇒ **TO-DO NEXT:** In the Jupyter Notebook accompanying this lesson, you will find an example on using the binomial model to price American options.

⇒ In the next lesson, we will revisit delta hedging for the specific context of American derivatives.

DERIVATIVE PRICING

MODULE 2 | LESSON 1

INTRO TO AMERICAN OPTIONS

Reading Time	60 minutes
Prior Knowledge	Binomial model, Option pricing
Keywords	American options, Early-exercise, Volatility matching

In this lesson, we are going to introduce the American payoff and how to implement the pricing of an American option with Python. Let's start importing some of the libraries that we will need to use down the road:

```
In [1]: import numpy as np
```

1. American Options Payoff

An American option is a style of options contract that allows holders to exercise their rights at any time before and on the expiration date. Thus, American-style options allow investors to capture profit as soon as the stock price moves favorably.

The value of this option, if it is left alive until maturity, is the same as its European counterpart. On the other hand, the value of the option if it is exercised is given by:

$$\max(0, S_t - K)$$

in the call case and,

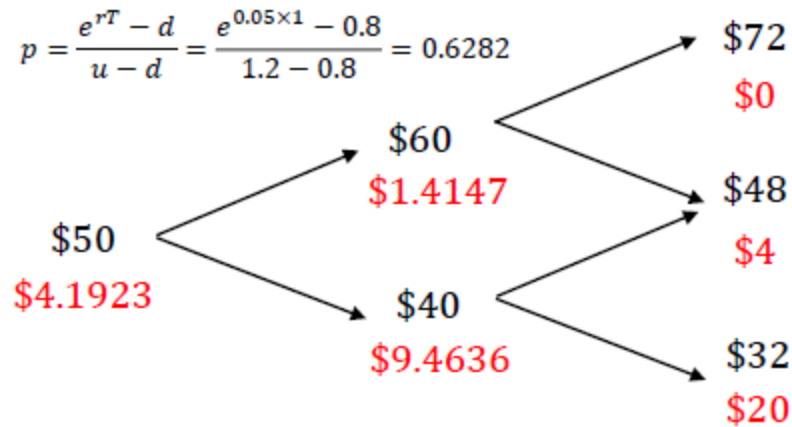
$$\max(0, K - S_t)$$

in the put case.

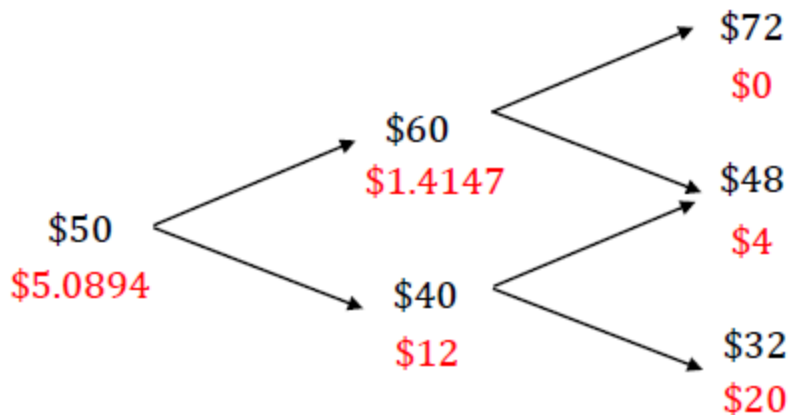
Thus, the **first step** consists of computing the European option prices for each node in the tree.

Second, for each node, we get the maximum value between the European option price and the payoff from early exercise.

For instance, if we follow the slide example:



The above figure shows the European option price for each node. In the **second step**, we decide whether to exercise the option in each node. Thus, in the next figure, we are going to keep the maximum value (in each node) between the European option price and the payoff from early exercise. See figure below:



The node that changes from the European option price to the American price is node {d}.

Why? It is simple: We are comparing the European option price, \$9.4636, with the payoff from early exercise, which is given by:

$$K - S = \$52 - \$40 = \$12 > 9.4636$$

Just for completeness, in the other node in which we should make a decision, {u}, the payoff from early exercise is lower than the European option price:

$$K - S = \$52 - \$60 = -\$8 < \$1.4247$$

```
In [2]: def american_option(S_ini, K, T, r, u, d, N, opttype):
    dt = T / N # Define time step
    p = (np.exp(r * dt) - d) / (u - d) # risk neutral probs
    C = np.zeros([N + 1, N + 1]) # call prices
    S = np.zeros([N + 1, N + 1]) # underlying price

    for i in range(0, N + 1):
        S[N, i] = S_ini * (u ** (i)) * (d ** (N - i))
        if opttype == "C":
            C[N, i] = max(S[N, i] - K, 0)
        else:
            C[N, i] = max(K - S[N, i], 0)

    for j in range(N - 1, -1, -1):
        for i in range(0, j + 1):
            C[j, i] = np.exp(-r * dt) * (
                p * C[j + 1, i + 1] + (1 - p) * C[j + 1, i]
            ) # Computing the European option prices
            S[j, i] = (
                S_ini * (u ** (i)) * (d ** (j - i))
            ) # Underlying evolution for each node
            if opttype == "C":
                C[j, i] = max(
                    C[j, i], S[j, i] - K
                ) # Decision between the European option price and the payoff from
            else:
                C[j, i] = max(
                    C[j, i], K - S[j, i]
                ) # Decision between the European option price and the payoff from

    return C[0, 0], C, S
```

2. Numerical Interpretation

Thus, for an American call, the value of the American option at a node is given by:

$$C(S, K, t) = \max(S - K, e^{-rt} [C(uS, t + dt)p^* + C(dS, t + dt)(1 - p^*)])$$

where,

$$e^{-rt} [C(uS, t + dt)p^* + C(dS, t + dt)(1 - p^*)]$$

represent the expected discounted value of "leaving alive" the option for another period (European option price). In other words,

$$e^{-rt} E^*[C(t + dt)] = e^{-rt} [C(uS, t + dt)p^* + C(dS, t + dt)(1 - p^*)]$$

Thus, in each node, we will make the decision of exercising the option only if the expected discounted value of the call in the next period is less than the value that we get if we decide to exercise the option.

Finally, let's try to reproduce our "toy-model" using the code. Remember the "toy-model" parameters (European Put Option):

$$u = 1.2, d = 0.8, T = 2 = N, dt = 1, r = 5\%, K = 52, S_0 = 50$$

```
In [3]: option_price, C, S = american_option(145, 100, 100, 0.1, 1.2, 0.8, 100, "P")
```

First, we check the underlying evolution...

```
In [4]: S
```

```
Out[4]: array([[1.45000000e+02, 0.00000000e+00, 0.00000000e+00, ...,
               0.00000000e+00, 0.00000000e+00, 0.00000000e+00],
              [1.16000000e+02, 1.74000000e+02, 0.00000000e+00, ...,
               0.00000000e+00, 0.00000000e+00, 0.00000000e+00],
              [9.28000000e+01, 1.39200000e+02, 2.08800000e+02, ...,
               0.00000000e+00, 0.00000000e+00, 0.00000000e+00],
              ...,
              [4.61515963e-08, 6.92273945e-08, 1.03841092e-07, ...,
               8.33930993e+09, 0.00000000e+00, 0.00000000e+00],
              [3.69212771e-08, 5.53819156e-08, 8.30728734e-08, ...,
               6.67144795e+09, 1.00071719e+10, 0.00000000e+00],
              [2.95370217e-08, 4.43055325e-08, 6.64582987e-08, ...,
               5.33715836e+09, 8.00573754e+09, 1.20086063e+10]])
```

Second, we check the option price in each node...

```
In [5]: C
```

```
Out[5]: array([[6.62571813e-01, 0.00000000e+00, 0.00000000e+00, ...,
               0.00000000e+00, 0.00000000e+00, 0.00000000e+00],
              [2.26752458e+00, 2.55184097e-01, 0.00000000e+00, ...,
               0.00000000e+00, 0.00000000e+00, 0.00000000e+00],
              [7.68873280e+00, 8.95515925e-01, 9.13844093e-02, ...,
               0.00000000e+00, 0.00000000e+00, 0.00000000e+00],
              ...,
              [1.00000000e+02, 9.99999999e+01, 9.99999999e+01, ...,
               0.00000000e+00, 0.00000000e+00, 0.00000000e+00],
              [1.00000000e+02, 9.99999999e+01, 9.99999999e+01, ...,
               0.00000000e+00, 0.00000000e+00, 0.00000000e+00],
              [1.00000000e+02, 1.00000000e+02, 9.99999999e+01, ...,
               0.00000000e+00, 0.00000000e+00, 0.00000000e+00]])
```

Thus, finally, the price of the American put is given by...

```
In [6]: print(option_price)
```

```
0.662571812972355
```

3. Matching Volatility in the American Option Function

Finally, in the next function, we compute the parameters "u" and "d" as a function of the underlying volatility (see Module 1, Lesson 4).

```
In [7]: def american_option_vol(S_ini, K, T, r, sigma, N, opttype):
    dt = T / N # Define time step
    u = np.exp(sigma * np.sqrt(dt)) # Define u
    d = np.exp(-sigma * np.sqrt(dt)) # Define d
    p = (np.exp(r * dt) - d) / (u - d) # risk neutral probs
    C = np.zeros([N + 1, N + 1]) # call prices
    S = np.zeros([N + 1, N + 1]) # underlying price

    for i in range(0, N + 1):
        S[N, i] = S_ini * (u ** (i)) * (d ** (N - i))
        if opttype == "C":
            C[N, i] = max(S[N, i] - K, 0)
        else:
            C[N, i] = max(K - S[N, i], 0)

    for j in range(N - 1, -1, -1):
        for i in range(0, j + 1):
            C[j, i] = np.exp(-r * dt) * (
                p * C[j + 1, i + 1] + (1 - p) * C[j + 1, i]
            ) # Computing the European option prices
            S[j, i] = (
                S_ini * (u ** (i)) * (d ** (j - i))
            ) # Underlying evolution for each node
            if opttype == "C":
                C[j, i] = max(
                    C[j, i], S[j, i] - K
                ) # Decision between the European option price and the payoff from
            else:
                C[j, i] = max(
                    C[j, i], K - S[j, i]
                ) # Decision between the European option price and the payoff from

    return C[0, 0], C, S
```

To conclude this example, let's try to get the price of an American option with the same parameters as the European call option of Module 1 (Lesson 4). That is:

$$K = 90, r = 0\%, T = 1, \sigma = 0.3$$

```
In [8]: price, C, S = american_option_vol(100, 90, 10, 0, 0.3, 10, "C")
print(price)
```

39.899240766330934

Finally, let's now see what call option ($K = 90, r = 0\%, T = 1, \sigma = 0.3$) price we obtain for different N's:

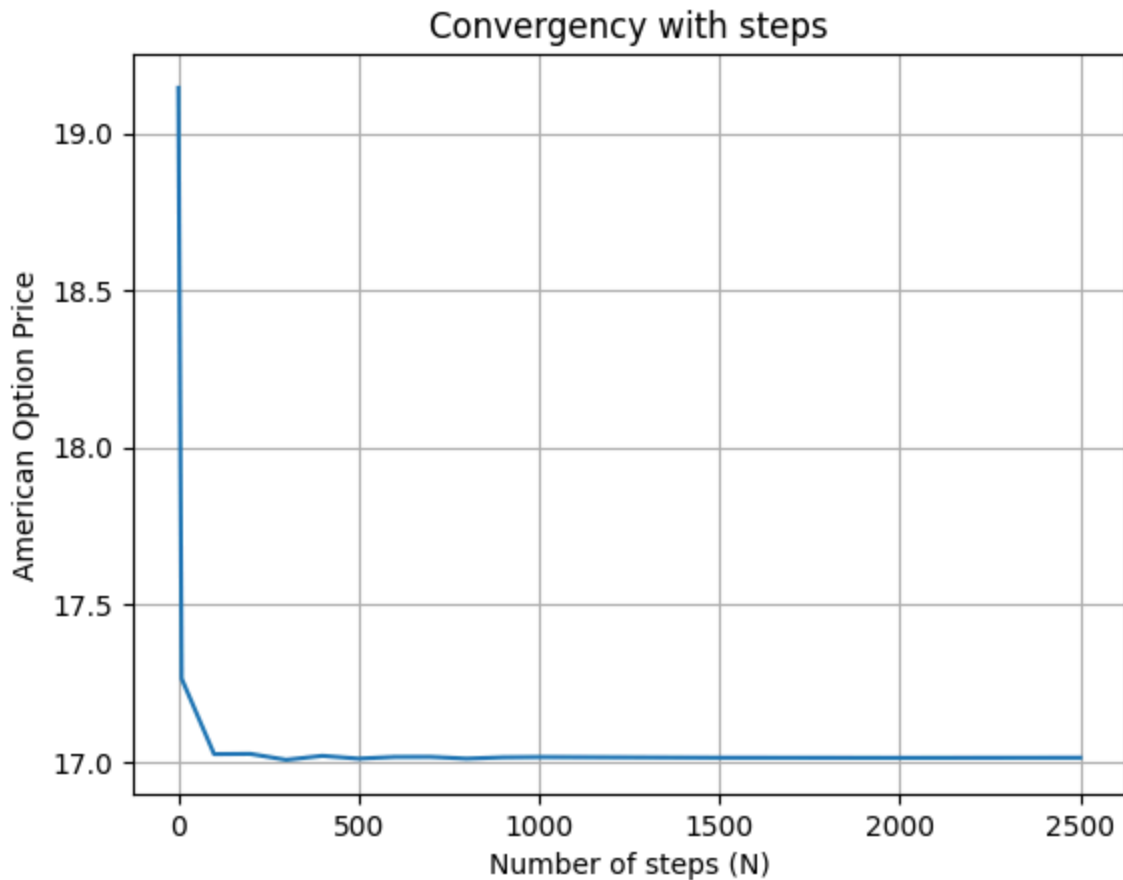
```
In [9]: price_array = []
for N in [1, 10, 100, 200, 300, 400, 500, 600, 700, 800, 900, 1000, 1500, 2000, 2500]:
    call_price, C, S = american_option_vol(100, 90, 1, 0, 0.3, N, "C")
```

```
price_array.append(call_price)
print("With N = {:3d}, the price is {:.2f}".format(N, call_price))
```

```
With N = 1, the price is 19.14
With N = 10, the price is 17.26
With N = 100, the price is 17.02
With N = 200, the price is 17.03
With N = 300, the price is 17.01
With N = 400, the price is 17.02
With N = 500, the price is 17.01
With N = 600, the price is 17.02
With N = 700, the price is 17.02
With N = 800, the price is 17.01
With N = 900, the price is 17.01
With N = 1000, the price is 17.02
With N = 1500, the price is 17.01
With N = 2000, the price is 17.01
With N = 2500, the price is 17.01
```

```
In [10]: import matplotlib.pyplot as plt
```

```
N = [1, 10, 100, 200, 300, 400, 500, 600, 700, 800, 900, 1000, 1500, 2000, 2500]
plt.plot(N, np.array(price_array))
plt.title("Convergency with steps")
plt.xlabel("Number of steps (N)")
plt.ylabel("American Option Price")
plt.grid(True)
plt.show()
```



4. Conclusion

In this lesson, we have gone over the basics of American option pricing and the fundamental differences compared to European options. In the next lesson, we will revisit the concept of delta hedging in the context of American derivatives.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use.
Redistribution or publication of this material is strictly prohibited.

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Schumacher, J. M. *Introduction to Financial Derivatives*. Open Press TiU.

<https://research.tilburguniversity.edu/en/publications/introduction-to-financial-derivatives>

- **Required pages:** Chapter 2, Section 2.2
- **Estimated reading time:** 30 minutes

NOTE: Content presented in the Required Readings and Lesson Notes will be tested in all assignments, including Quizzes.

LESSON 2

DYNAMIC DELTA HEDGING

Derivative Pricing - Module 2



Outline

- ▶ Revisiting Delta Hedging
- ▶ Dynamic Delta Hedging
- ▶ Delta Hedging in American vs. European Options

Revisiting Delta Hedging

In Module 1, we took a look at the delta measure and highlighted its importance to construct risk-neutral portfolios (i.e., *delta-neutral*).

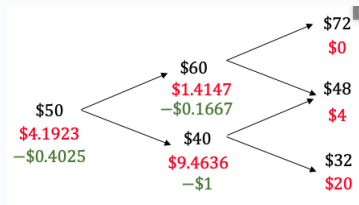
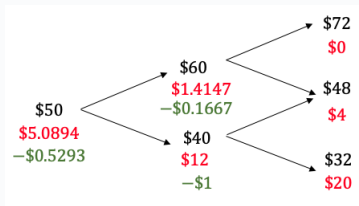
In this lesson, we are revisiting the concept with specific application to American options.

- ▶ Delta measures how the option price changes when underlying price moves: $\frac{\partial C}{\partial S}$
- ▶ This is still (and always!) the case for American options.
- ▶ Only change is payoff for the American derivative may be different from European.
- ▶ Importantly for American derivatives, hedging using only terminal prices makes less sense.
- ▶ Thus, there will be a trade-off between the number of steps in the tree (and dt) and the ability to continuously hedge our delta exposure $\Rightarrow$ **Dynamic Hedging**

Let's see an example of delta hedging for the American Put option we covered in lesson 1 and compare that to the hedging of the European counterpart.

Delta Hedging

Here, you have the computation of delta (in green) for American (above) and European (below) put options:



Dynamic Delta Hedging

From the latter example, there are many things worth noting:

- ▶ Delta hedging is costlier for American options (if early exercise optimal)
 - ⇒ Does this make sense?
Yes (look at prices)
- ▶ Remember the example from Module 1 on call option delta hedging positions:
 - ⇒ Here, positions are the other way around (i.e. $\Delta < 0$), why? → Put!
- ▶ Step-by-step adjustment of our hedging is key for American options.
 - ⇒ Dynamic Delta Hedging:
 - Replication portfolio at all times.
 - Especially relevant in a continuous-time framework → Black-Scholes

There will be a trade-off between the number of steps we consider in our tree and the ability to fully hedge our delta-risk.

Summary of Lesson 2

In lesson 2, we have covered several important concepts:

- ▶ Revisiting delta hedging for American options.
- ▶ Delta hedging for American vs. European options: consequences of early exercise.
- ▶ Importance of dynamic delta hedging for American options.

⇒ **TO-DO NEXT:** In the **Jupyter Notebook accompanying this lesson**, you will find an example of how to perform delta hedging in American derivatives and the differences compared to their European counterparts.

⇒ In the next lesson, we will dig deeper into the challenges in applying a simple framework like the binomial model to more complex option payoffs (e.g., path-dependent).

Lesson 2 - Dynamic Delta Hedging

February 8, 2026

0.1 DERIVATIVE PRICING

MODULE 2 | LESSON 2

1 DYNAMIC DELTA HEDGING

Reading Time	40 minutes
Prior Knowledge	Pricing in the Binomial model, Characteristics of American options, Delta hedging
Keywords	Dynamic delta hedging, American options

In the previous lesson, we introduced American options, whose main feature is the embedded possibility of early exercise of the option prior to maturity. This is going to be important for pricing but also for hedging purposes. In the context of European options, we could delta-hedge our portfolio by simply looking at terminal prices. This is no longer an option for American derivatives, which require dynamic delta hedging for every step in the tree. In this lesson, we will take a closer look at how to perform dynamic hedging.

As usual, let's start by importing the numpy library:

```
[1]: import numpy as np
```

1.1 1. Delta Hedging in the Binomial Tree: American Options

In the following snippet, you have the code for a general function to price a European option using a binomial tree methodology. The function, as you can see, is able to compute the evolution of the underlying asset S_t , the price of the option at each node of the tree, C_t , and also the Δ_t (number of shares in terms of exposure) in each node of the tree.

```
[2]: def american_option(S_ini, K, T, r, u, d, N, opttype):  
    dt = T / N # Define time step  
    p = (np.exp(r * dt) - d) / (u - d) # risk neutral probs  
    C = np.zeros([N + 1, N + 1]) # call prices  
    S = np.zeros([N + 1, N + 1]) # underlying price  
    Delta = np.zeros([N, N]) # delta
```

```

for i in range(0, N + 1):
    S[N, i] = S_ini * (u ** (i)) * (d ** (N - i))
    if opttype == "C":
        C[N, i] = max(S[N, i] - K, 0)
    else:
        C[N, i] = max(K - S[N, i], 0)

for j in range(N - 1, -1, -1):
    for i in range(0, j + 1):
        C[j, i] = np.exp(-r * dt) * (
            p * C[j + 1, i + 1] + (1 - p) * C[j + 1, i]
        ) # Computing the European option prices
        S[j, i] = (
            S_ini * (u ** (i)) * (d ** (j - i))
        ) # Underlying evolution for each node
        if opttype == "C":
            C[j, i] = max(
                C[j, i], S[j, i] - K
            ) # Decision between the European option price and the payoff
            ↪from early-exercise
        else:
            C[j, i] = max(
                C[j, i], K - S[j, i]
            ) # Decision between the European option price and the payoff
            ↪from early-exercise

        Delta[j, i] = (C[j + 1, i + 1] - C[j + 1, i]) / (
            S[j + 1, i + 1] - S[j + 1, i]
        ) # Computing the delta for each node

return C[0, 0], C, S, Delta

```

```
[3]: price, C, S, delta = american_option(45, 100, 5, 0, 1.5, 1 / 1.5, 5, "C")
```

```
[4]: price
```

```
[4]: np.float64(6.4592000000000002)
```

```
[5]: delta
```

```
[5]: array([[0.34208, 0., 0., 0., 0.],
          [0.1328, 0.4816, 0., 0., 0.],
          [0., 0.22133333, 0.65511111, 0., 0.],
          [0., 0., 0.36888889, 0.84592593, 0.],
          [0., 0., 0., 0.61481481, 1.]])
```

Let's now compare this delta hedging with the one from the European option with the same characteristics. For that, we will reuse some of the code from Module 1: All rights reserved WQU

```
[6]: def european_option(S_ini, K, T, r, u, d, N, opttype):
    dt = T / N # Define time step
    p = (np.exp(r * dt) - d) / (u - d) # risk neutral probs
    C = np.zeros([N + 1, N + 1]) # call prices
    S = np.zeros([N + 1, N + 1]) # underlying price
    Delta = np.zeros([N, N]) # delta

    for i in range(0, N + 1):
        S[N, i] = S_ini * (u ** (i)) * (d ** (N - i))
        if opttype == "C":
            C[N, i] = max(S[N, i] - K, 0)
        else:
            C[N, i] = max(K - S[N, i], 0)

    for j in range(N - 1, -1, -1):
        for i in range(0, j + 1):
            C[j, i] = np.exp(-r * dt) * (
                p * C[j + 1, i + 1] + (1 - p) * C[j + 1, i]
            ) # Computing the European option prices
            S[j, i] = (
                S_ini * (u ** (i)) * (d ** (j - i))
            ) # Underlying evolution for each node

            Delta[j, i] = (C[j + 1, i + 1] - C[j + 1, i]) / (
                S[j + 1, i + 1] - S[j + 1, i]
            ) # Computing the delta for each node

    return C[0, 0], C, S, Delta
```

```
[7]: price_euro, C_euro, S_euro, delta_euro = european_option(
    50, 52, 5, 0.05, 1.2, 0.8, 5, "C"
)
delta_euro
```

```
[7]: array([[0.72579669, 0.          , 0.          , 0.          , 0.          ],
          [0.44672292, 0.83592034, 0.          , 0.          , 0.          ],
          [0.09194177, 0.58672101, 0.93425542, 0.          , 0.          ],
          [0.          , 0.12822237, 0.76764645, 1.          , 0.          ],
          [0.          , 0.          , 0.17881944, 1.          , 1.          ]])
```

```
[8]: price_euro
```

```
[8]: np.float64(13.37283461304853)
```

1.2 2. Conclusion

In this lesson, we have seen the importance of dynamic delta hedging in the context of American derivatives. This is an important concept as well for other path-dependent options, as we will see in the next lesson.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Schumacher, J. M. *Introduction to Financial Derivatives*. Open Press TiU.

<https://research.tilburguniversity.edu/en/publications/introduction-to-financial-derivatives>

- **Required pages:** Chapter 2, Section 2.3
- **Estimated reading time:** 25 minutes

NOTE: Content presented in the Required Readings and Lesson Notes will be tested in all assignments, including Quizzes.

LESSON 3

INTRO TO PATH-DEPENDENT OPTIONS

Derivative Pricing - Module 2



Outline

- ▶ Intro to Path-dependent options.
- ▶ Problems in Binomial model for path-dependent options.
- ▶ Potential solutions & computational costs' trade-off.

Path-dependent options

So far, we have used the binomial tree to price options with conventional payoffs:

$$\Rightarrow (S_t - K)^+ \text{ for call options and } (K - S_t)^+ \text{ for put options.}$$

However, one of the main advantages of derivative instruments is they're highly customizable for *ad hoc* needs:

$\Rightarrow$ Most clients will demand OTC instruments with **non-conventional payoffs**!

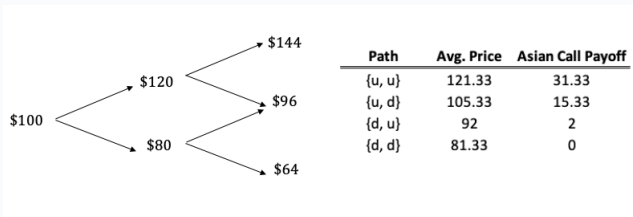
One example of such payoff is the **Asian option**:

- ▶ Payoff of Asian options is dependent on the arithmetic average of stock price:
 $\rightarrow (S_{Avg.} - K)^+$ for Asian Call, $(K - S_{Avg.})^+$ for Asian Put.
- ▶ This implies that we need to compute the average price, $S_{Avg.}$, for ALL paths
- ▶ While simple for the 'toy examples' that we have been using to illustrate concepts, there are a bunch of computational problems related to it
- ▶ We will use the Asian option to illustrate these, but note that problems can be extrapolated to any context with path-dependent options

Asian option on Binomial Model

Picture an Asian Call option under a binomial model with the following data:

$S_0 = 100$, $K = 90$, $T = 1$, $r = 0\%$, $u = 1.2$, $d = 0.8$, $N = 2 \rightarrow p = 0.5$



Values for the Asian Call option at step 1 in the tree will be:

$$C_1^u = 0.5 \times 31.33 + (1 - 0.5) \times 15.33 = \$23.33$$
$$C_1^d = 0.5 \times 2 + (1 - 0.5) \times 0 = \$1$$

So, value of the Asian option today will be:

$$C_0 = 0.5 \times 23.33 + (1 - 0.5) \times 1 = \$12.165$$

Asian option main takeaways

Takeaways from last example on Asian option:

- ▶ Note that here number of terminal prices $\neq$ number of option payoffs $\rightarrow$ Path matters
- ▶ To get option payoff we have to compute all paths $\rightarrow \#Paths = 2^N$
- ▶ Ideally, the more paths we compute the merrier
 - ▶ Improve accuracy?
 - ▶ Computational cost?
 - $\Rightarrow$ If you haven't yet, try running one of the previous codes for large N !
 - $\Rightarrow$ Picture you want $N = 100$ steps. That means 1.27×10^{30} paths!
- ▶ While conceptually it is easy to price Asian options using the binomial tree, implementing it in practice is way more difficult
- ▶ These problems are common to all types of path-dependent options/derivatives:
 - Barrier options; Lookback options; Binary options; ...

Summary of Lesson 3

In Lesson 3 we have seen some important concepts:

- ▶ Path-dependent options and unconventional payoffs
- ▶ Use of the binomial model to price options with path-dependent payoff
- ▶ Computational problems arising from using binomial model with path-dependent derivatives

⇒ **TO-DO NEXT:** This Lesson does not have an associated Jupyter Notebook. Move to Lesson 4, and its corresponding Notebook!

⇒ In the next lesson we introduce an alternative methodology to alleviate the computational problems embedded here: Monte-Carlo methods.

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Schumacher, J. M. *Introduction to Financial Derivatives*. Open Press TiU.

<https://research.tilburguniversity.edu/en/publications/introduction-to-financial-derivatives>

- **Required pages:** Chapter 2, Section 2.4
- **Estimated reading time:** 40 minutes

2. Schumacher, J. M. *Introduction to Financial Derivatives*. Open Press TiU.

<https://research.tilburguniversity.edu/en/publications/introduction-to-financial-derivatives>

- **Required pages:** Chapter 2, Section 2.5
- **Estimated reading time:** 30 minutes

NOTE: Content presented in the Required Readings and Lesson Notes will be tested in all assignments, including Quizzes.

LESSON 4

INTRODUCTION TO MONTE CARLO METHODS

Derivative Pricing - Module 2



Outline

- ▶ Introduction to Monte Carlo (MC) methods: Concept and intuition
- ▶ Monte Carlo method for pricing a European option
- ▶ Monte Carlo method for pricing an Asian option

Intuition behind Monte Carlo methods

- ▶ In Lesson 3, we illustrated the problems related to using the binomial model to price path-dependent derivatives like Asian options.
- ▶ In Lesson 4, we will develop the basic intuition for a method that allows us to overcome these problems: **Monte Carlo**.

⇒ What are Monte Carlo (MC) methods?

- MC methods rely on **simulations & random sampling** to obtain an approximate solution to a problem.
- MC methods can be used in any type of problem with probabilistic interpretation.
- **Intuition:** expected value approximated by empirical mean of simulated values (law of large numbers).
- Accuracy of the method (convergence to true value) increases with # of simulations.
- Again, we face the trade-off between computational costs vs. accuracy.

MC Example for European Option (plain vanilla)

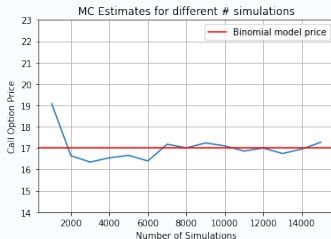
Let's start by checking the power of MC methods on a simple (plain vanilla) European call option.

(Calculations and code for all the following results are in the Jupyter Notebook accompanying this lesson.)

⇒ European call option with the same characteristics as in Module 1:

$S_0 = 100, K = 90, T = 1, \sigma = 0.3, r = 0\%, N = 2500$

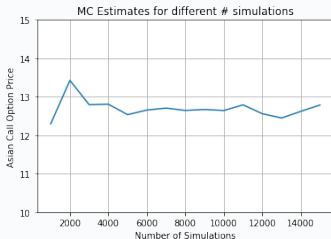
- ▶ In Lesson 1, under binomial model & $N = 2,500$ we got a *Call Price* = \$17.01
- ▶ Using MC methods:



MC example for path-dependent options

While the former example does not make a lot of sense in practice (i.e., it is much less costly to get the actual price from a binomial model), MC methods are very useful in practice for **path-dependent options**:

- ▶ e.g., for an **Asian call option** similar to the one from Lesson 3 with:
 $S_0 = 100$; $K = 90$; $r = 0\%$; $T = 1$; $\sigma = 0.3$; $N = 2,500$
- ▶ In this case, going over ALL the paths would be a never-ending task → We cannot compute ALL paths.
- ▶ We can, however, simulate some amount of paths randomly and use MC intuition to price the option.



Summary of Lesson 4

In Lesson 4, we have:

- ▶ (Informally) Introduced a powerful tool for option pricing: Monte Carlo methods.
- ▶ Examined the convergence of the Monte Carlo method with European call option prices.
- ▶ Discussed the usefulness of Monte Carlo methods for pricing path-dependent options.

⇒ **TO-DO NEXT:** You should have followed this lesson using the code in the accompanying Jupyter Notebook.

⇒ In the next module, we extend the binomial tree to recognize a higher variety of stock price movements and analyze the complications associated with more complex frameworks when it comes to replication.

Lesson 4 - Intro to Monte Carlo Methods

February 8, 2026

0.1 DERIVATIVE PRICING

MODULE 2 | LESSON 4

1 INTRO TO MONTE CARLO METHODS

Reading Time	60 minutes
Prior Knowledge	Binomial model, Option pricing, European options, American options, Asian options
Keywords	Monte-Carlo methods, Path-dependent options

—All rights reserved WQU WorldQuant University QQQQ

In Lesson 4 of Module 2, we will go through the basics of using Monte Carlo methods for derivative pricing. Ideally, you should look at this notebook while working with the corresponding video and slides, but you can also work on this on your own.

1.1 1. Monte Carlo Methods

Please note that we are introducing Monte Carlo methods informally here. Monte Carlo comes in handy in a lot of derivative pricing problems in real life, and we will cover this method thoroughly throughout the course. We believe this simple introduction will help down the road regarding basic intuition. The mapping of what the actual analytical computation does versus the Monte Carlo estimation will be very easy to see in this simple setting. In the future, it can be more complex though, so we believe it is best to introduce it here.

Let's start importing the numpy library:

```
[1]: import numpy as np
```

Next, we are going to develop a function, `call_option_mc`, that will use Monte Carlo-like methods to study the power of this technique.

The basic intuition of Monte Carlo is that by simulating a fair amount of outcomes randomly from the same distribution as our underlying process, we can obtain a solution close to the actual (full) process. The process will go as follows: - Select the distribution you are using for the underlying process: In our case, it is a binomial distribution for the stock price. - Simulate random realizations of such distribution: We use Python to choose a random number from a binomial distribution using

np.random.binomial. - Calculate option payoff for those simulated values and discount them so that you can confirm a price for the option under each simulated path. - Get the mean of those prices: By relying on the law of large numbers, when the amount of randomly generated values is high enough, the average of all converges to the true value of the options. Of course, the more simulations, the merrier.

Let's get to it with the European call option example:

1.2 2. European Option Payoff (Monte Carlo)

```
[2]: def call_option_mc(S_ini, K, T, r, sigma, N, M):
    dt = T / N # Define time step
    u = np.exp(sigma * np.sqrt(dt)) # Define u
    d = np.exp(-sigma * np.sqrt(dt)) # Define d
    p = (np.exp(r * dt) - d) / (u - d) # risk neutral probs
    C = np.zeros([M]) # call prices
    S = np.zeros([M, N + 1]) # underlying price
    S[:, 0] = S_ini

    for j in range(0, M):
        random = np.random.binomial(
            1, p, N + 1
        ) # We sample random realizations for the paths of the tree under a
        ↪ binomial distribution
        for i in range(1, N + 1):
            if random[i] == 1:
                S[j, i] = S[j, i - 1] * u
            else:
                S[j, i] = S[j, i - 1] * d

        C[j] = np.exp(-r * T) * max(S[j, N] - K, 0)

    return S, C
```

Now that we have our function, let's check its performance by comparing it to the European call option we priced in Module 1. If you remember, these were the characteristics of the option: - $S_0 = 100$ - $K = 90$ - $T = 1$ - $r = 0\%$ - $\sigma = 0.3$

If you remember, when we increase the number of paths in the binomial tree, N , the price of the option converged to \$17.01 for $N = 2,500$.

So, let's see how our (quasi) Monte Carlo algorithm performs for $N = 2,500$ using 15,000 simulations (paths):

```
[3]: S, C = call_option_mc(
    100, 90, 1, 0, 0.3, 2500, 15000
) # Prices 15000 different simulations
```

Now the variable C contains the 15,000 different prices from the different simulations performed. In order to get the final value for the call option proxied with Monte Carlo, we just average all

these values:

```
[4]: print(np.mean(C))
```

```
16.633446841194928
```

As you can see, the value is pretty close to the one we got from the workout on the full binomial model of \$17.01.

But let's dig a little deeper into the convergence to this value. We will now check how we start approximating to this value as we increase the number of simulations:

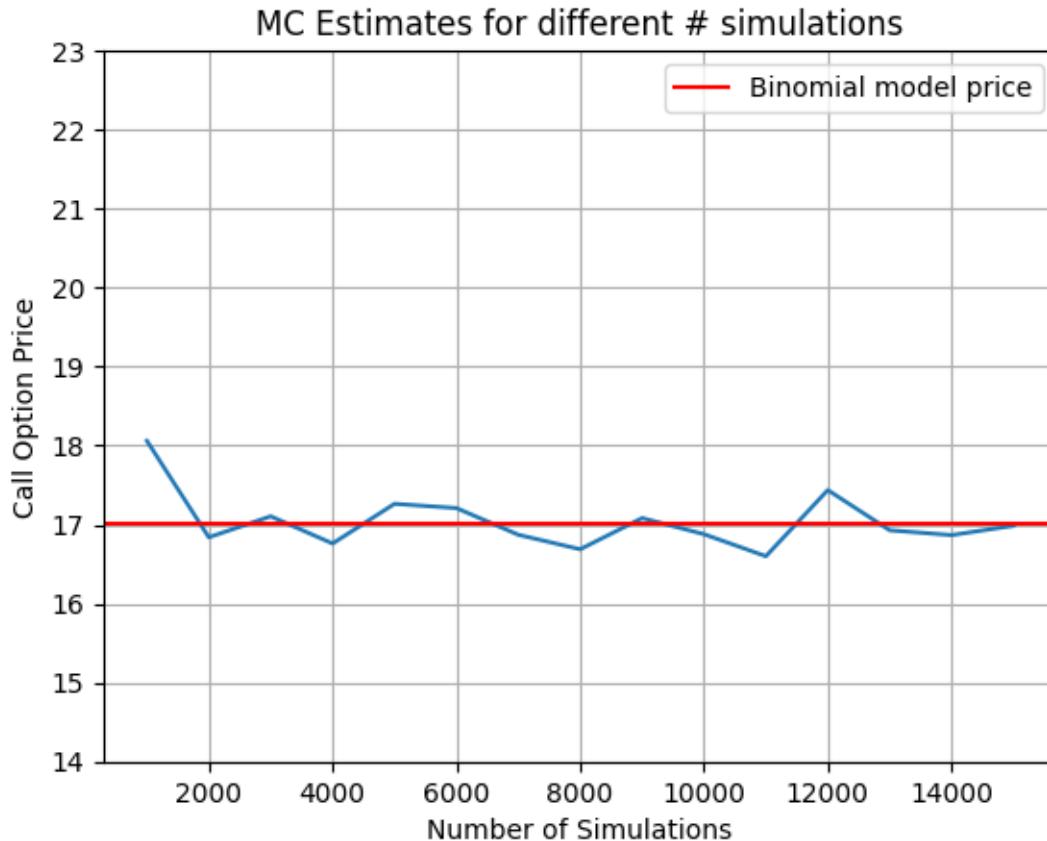
```
[5]: M = np.arange(1000, 16000, 1000) # Different number of simulations to be
      ↪ considered
      call_price = []
```

Warning: The following code will take several minutes to run!

```
[6]: for i in range(len(M)):
      S, C = call_option_mc(100, 90, 1, 0, 0.3, 2500, M[i])
      call_price.append(np.mean(C))
```

```
[7]: import matplotlib.pyplot as plt

      plt.plot(M, call_price)
      plt.axhline(y=17.01, color="r", linestyle="--", label="Binomial model price")
      plt.ylim([14, 23])
      plt.title("MC Estimates for different # simulations")
      plt.xlabel("Number of Simulations")
      plt.ylabel("Call Option Price")
      plt.grid(True)
      plt.legend()
      plt.show()
```



While the past exercise may seem in vain, given that we can simply use the binomial model to price the option, it gives us a pretty good idea of how the Monte Carlo method works.

Understanding Monte Carlo methods, thus, will help us provide efficient solutions/pricing in more problematic scenarios, as was the case with the Asian option. In this framework, calculating ALL the potential payoffs of the option for $N = 2,500$ would mean we have to deal with $2^N = 2^{2500}$ paths. That is virtually impossible and inefficient.

Hence, Monte Carlo will come in handy in these type of situations for path-dependent derivatives. Let's now build up a function, `asian_option_mc`, to price an Asian option with the same characteristics using this technique:

1.3 3. Asian Option Payoff (Monte Carlo)

```
[8]: def asian_option_mc(S_ini, K, T, r, sigma, N, M):
    dt = T / N # Define time step
    u = np.exp(sigma * np.sqrt(dt)) # Define u
    d = np.exp(-sigma * np.sqrt(dt)) # Define d
    p = (np.exp(r * dt) - d) / (u - d) # risk neutral probs
    Asian = np.zeros([M]) # Asian prices
    S = np.zeros([M, N + 1]) # underlying price
```

```

S[:, 0] = S_ini

for j in range(0, M):
    random = np.random.binomial(1, p, N + 1)
    Total = S_ini
    for i in range(1, N + 1):
        if random[i] == 1:
            S[j, i] = S[j, i - 1] * u
            Total = Total + S[j, i]
        else:
            S[j, i] = S[j, i - 1] * d
            Total = Total + S[j, i]

    Asian[j] = np.exp(-r * T) * max(Total / (N + 1) - K, 0)

return S, Asian

```

As before, let's check the prices for the different paths:

```
[9]: S, Asian = asian_option_mc(100, 90, 1, 0, 0.3, 2500, 10000)
```

And average them to get a final estimate:

```
[10]: print(np.mean(Asian))
```

12.65477784654346

Let's also study the convergence of the methods in a similar fashion as before:

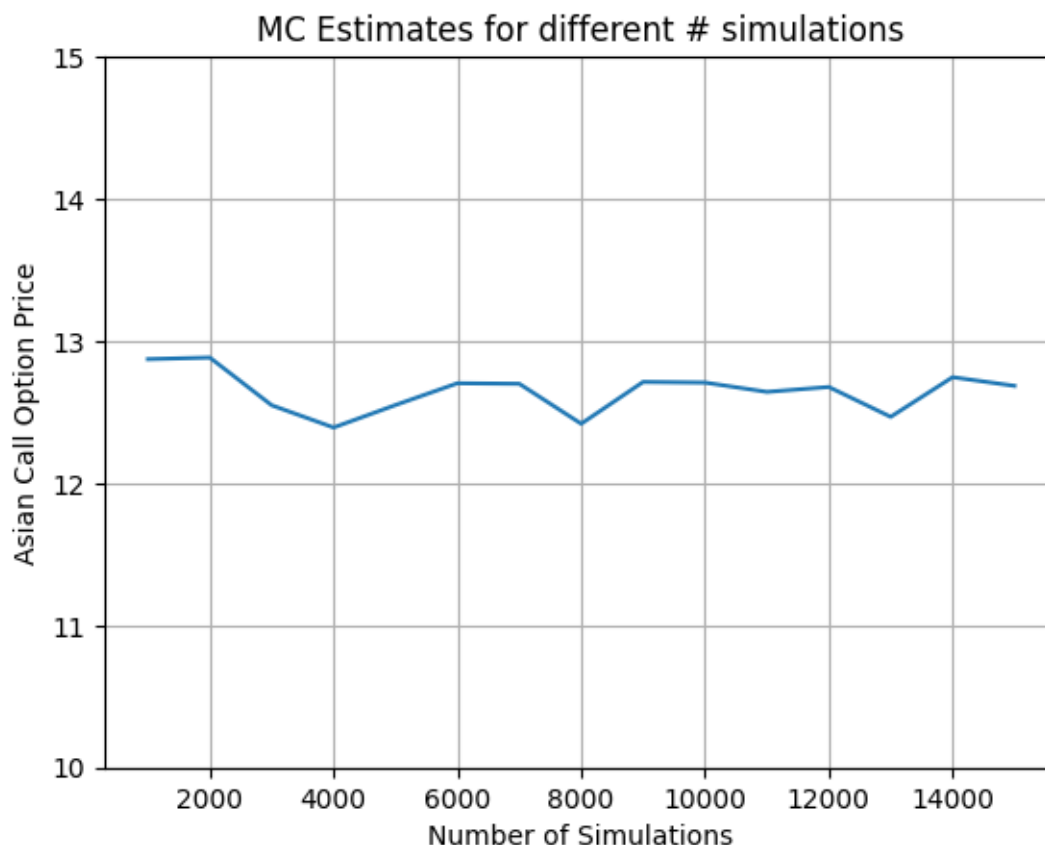
```
[11]: M = np.arange(1000, 16000, 1000)
asian_price = []
```

Warning: The following code will take several minutes to run!

```
[12]: for i in range(len(M)):
    S, Asian = asian_option_mc(100, 90, 1, 0, 0.3, 2500, M[i])
    asian_price.append(np.mean(Asian))
```

```
[13]: import matplotlib.pyplot as plt

plt.plot(M, asian_price)
plt.ylim([10, 15])
plt.title("MC Estimates for different # simulations")
plt.xlabel("Number of Simulations")
plt.ylabel("Asian Call Option Price")
plt.grid(True)
plt.show()
```



1.4 4. Conclusion

All in all, you have seen firsthand how mastering Monte Carlo methods can be a fundamental ally when it comes to pricing complex derivatives for which it is too costly (or impossible) to actually compute all the potential outcomes of the underlying process.

Please note once again that this lesson introduces Monte Carlo from an informal perspective and only with the goal of simplifying your understanding of it. We will dig deeper into this in future modules.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.